



深圳大学
Shenzhen University

互联网编程

第3章

线程

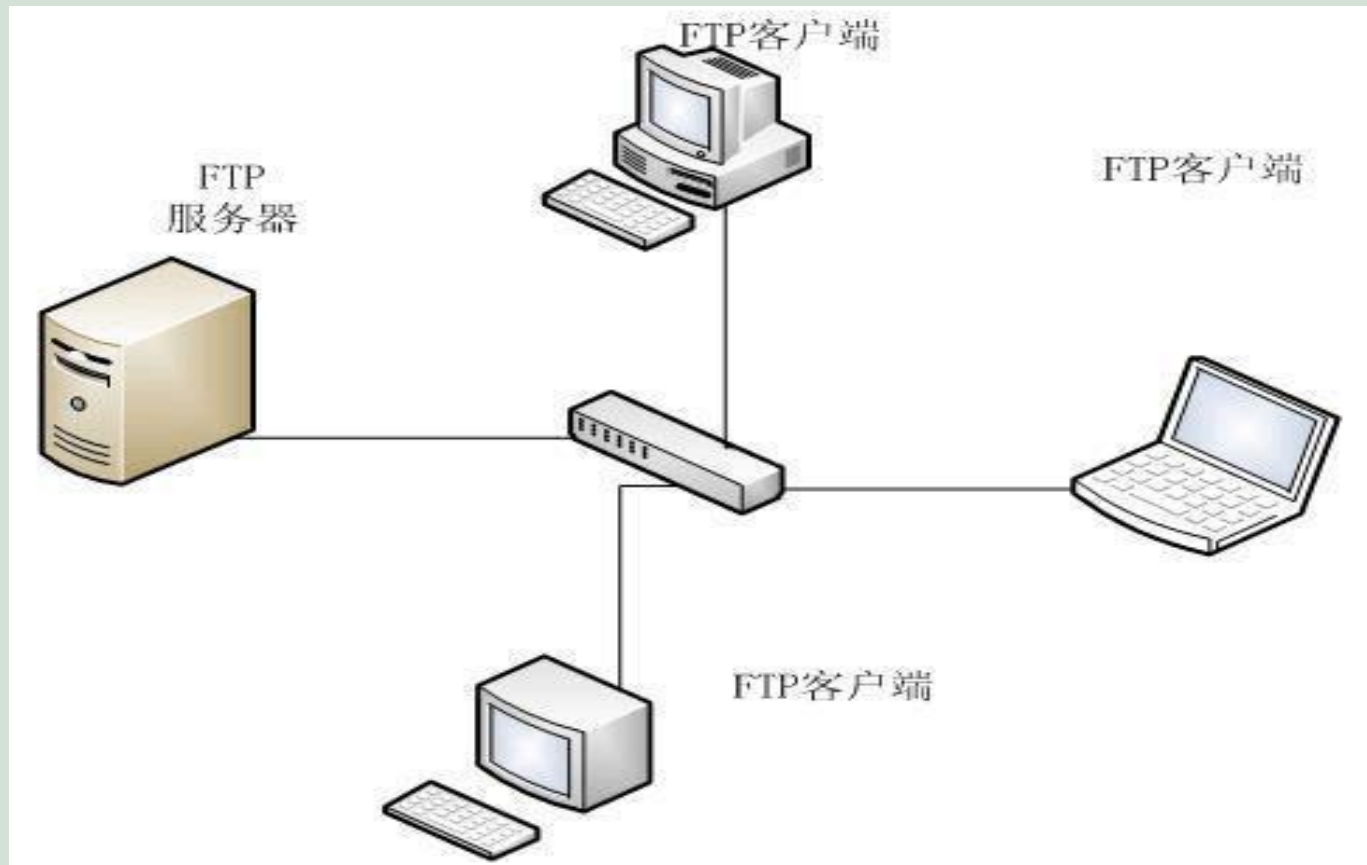
深圳大学计算机与软件学院

主要内容

- 网络编程应用场景
- 应用程序的并发性发展
- **Java**多线程编程技术及例子
- 从线程返回信息及不稳定性问题
- 轮询法编程及例子
- 回调法编程及例子

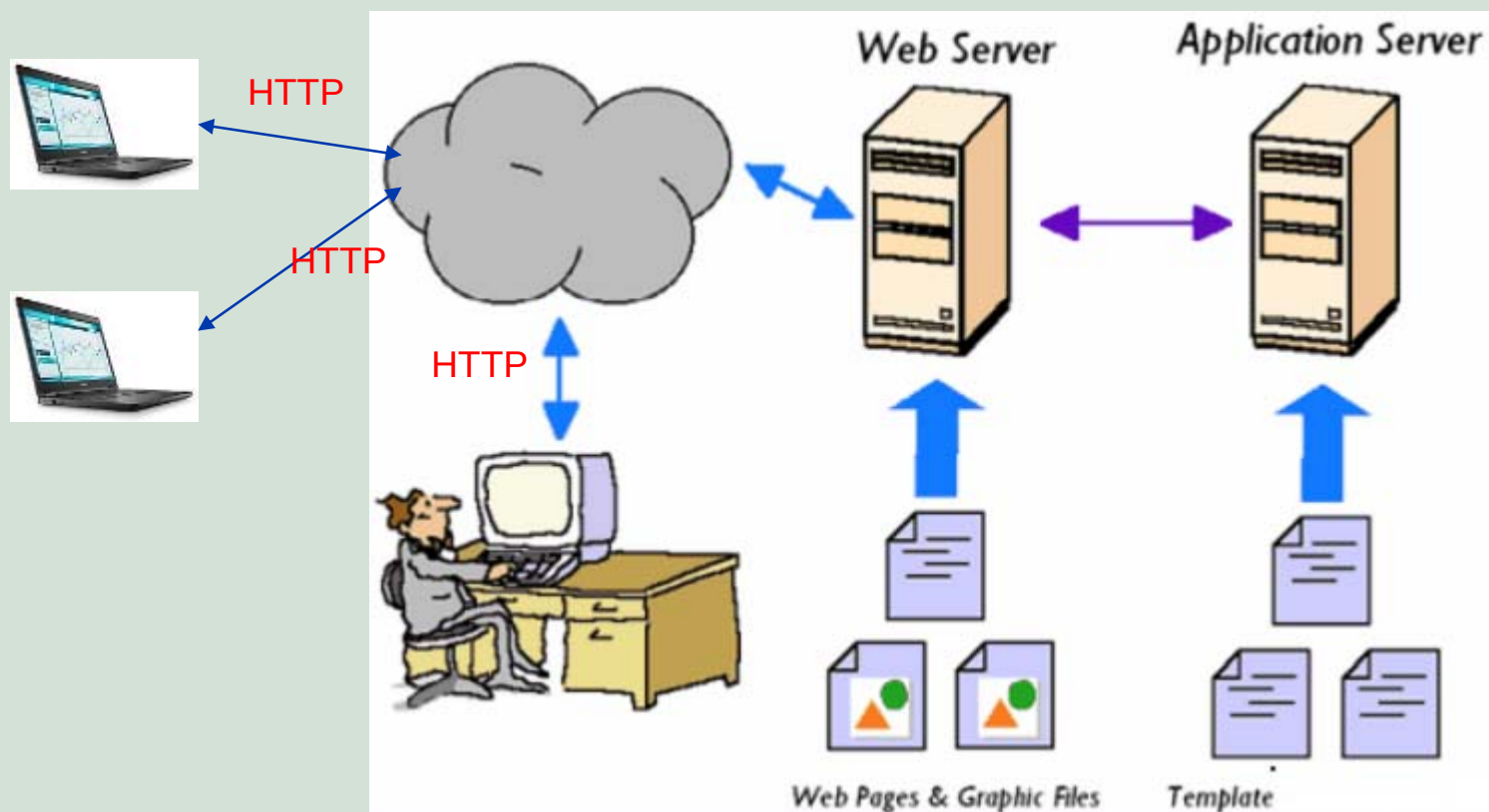


网络编程应用场景



FTP应用编程: FTP客户端来一个连接, FTP服务器创建一个进程处理连接
存在问题: 多用户并发情况下, 多进程易拖跨FTP服务器, FTP服务器响应性能极差.

网络编程应用场景



Http应用编程：并发多个HTTP连接请求,也会造成web服务器性能下降。如何编程保证web服务器性能上的可靠性？

多进程----多线程-----线程池

多线程的局限性：当并发线程数达到4000-20000时，大多数jvm会因内存耗尽而无法承受。

从多进程到多线程

应用系统的
多用户并发
访问特点

➤ 提升并发量的方法

过去：以进程为单位

- 多进程：支持几百个进程
- 进程池：支持上千个进程

当前：以线程为单位

- 多线程：支持几千个线程
- 线程池：支持上万个线程



回顾java多线程编程

➤ 1.创建启动线程对象

```
Thread t = new Thread( );  
t.start( );
```

➤ 2.让线程完成的任务放在哪儿？ 重写run（）方法

创建线程类重写run
方法做“我想让线程
干的事”

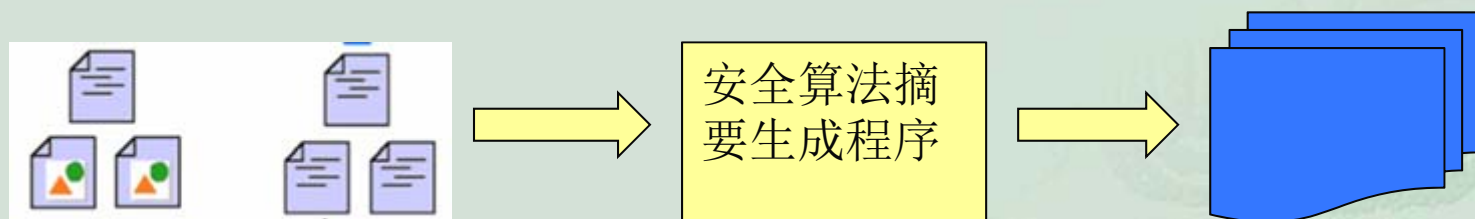
派生Thread

实现Runnable接口



编程示范例子

- 1.文件安全算法摘要生成编程：计算多个文件的安全算法（SHA）摘要，对每件文件应用安全散列算法（SHA）计算给出该文件的安全算法摘要。



思考：影响该程序性能的主要因素是？

访问文件的I/O读写

程序的运行速度 >> 文件数据读写速度

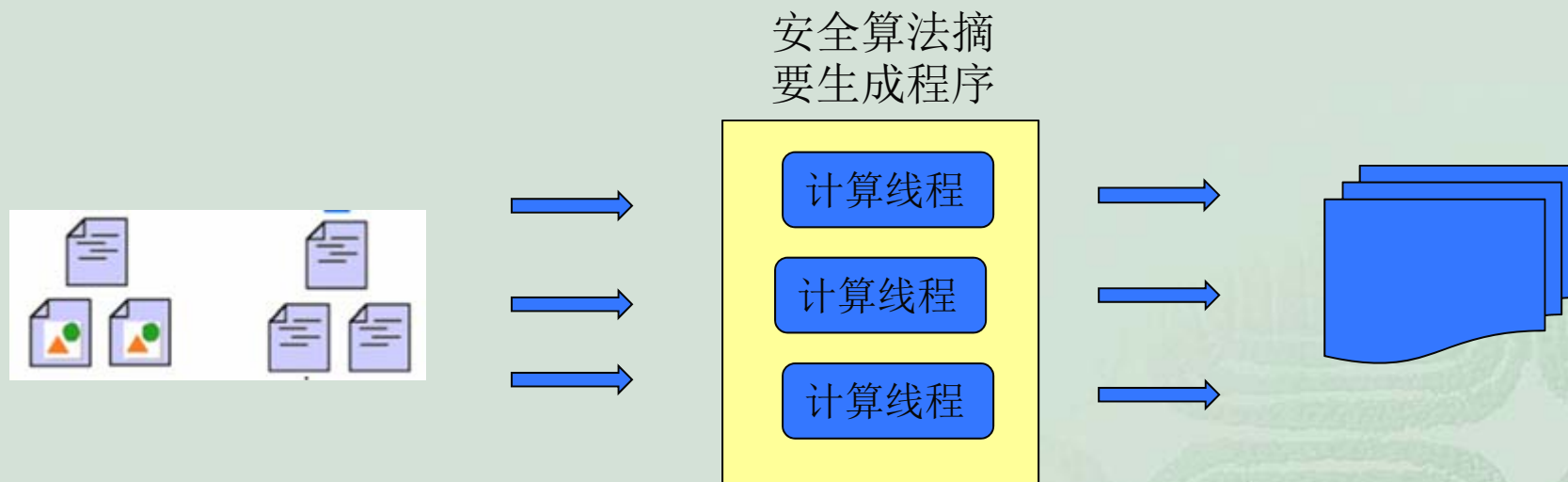
等待、程序阻塞，拖慢程序执行速度

使用多线程编程，缓解程序等待情况。



编程示范例子

- 使用派生Thread方式编写线程。



编码例子演示: ch3-1 DigestThread.java
对磁盘中digest(C:\temp\ch3data\digest)目录下的三个
文件生成安全算法摘要。
1.txt , build.xml, 2.txt

ch3-1 DigestThread.java

DigestThread.java

```
1 package ch3;
2 import java.io.*;
3 import java.security.*;
4 import javax.xml.bind.*; // for DatatypeConverter; requires Java 6 or JAXB 1.0
5 public class DigestThread extends Thread {
6     private String filename;
7     public DigestThread(String filename) {
8         this.filename = filename;
9     }
10    @Override
11    public void run() {
12        try {
13            System.out.println("Thread processing file is : ");
14            FileInputStream in = new FileInputStream(filename);
15            MessageDigest sha = MessageDigest.getInstance("SHA-256");
16            DigestInputStream din = new DigestInputStream(in, sha);
17            while (din.read() != -1) ;
18            din.close();
19            byte[] digest = sha.digest();
20
21            StringBuilder result = new StringBuilder(filename);
22            result.append(": ");
23            result.append(DatatypeConverter.printHexBinary(digest));
24            System.out.println(result);
25        } catch (IOException ex) {
26            System.err.println(ex);
27        } catch (NoSuchAlgorithmException ex) {
28            System.err.println(ex);
29        }
30    }
31
32    public static void main(String[] args) {
33        System.out.println("args is : " + args);
34        for (String filename : args) {
35            Thread t = new DigestThread(filename);
36            t.start();
37        }
38    }
39 }
```

运行参数:

C:\temp\ch3data\digest\1.txt

C:\temp\ch3data\digest\build.xml

C:\temp\ch3data\digest\2.txt

编程示范例子

- 使用实现Runnable接口方式编写线程。

自定义Runnable线程类（实现Runnable）可以作为其他类的子类。要执行的线程任务放在run()方法中。

```
Thread t = new Thread(myRunnableObject);  
t.start;
```

编码例子演示： ch3-2 DigestRunnable.java



ch3-2 DigestRunnable.java

```
DigestRunnable.java ✕
3 import java.io.*;
4 import java.security.*;
5 import javax.xml.bind.*; // for DatatypeConverter; requires Java 6 or JAXB 1.0
6 public class DigestRunnable implements Runnable {
7     private String filename;
8     public DigestRunnable(String filename) {
9         this.filename = filename;
10    }
11    @Override
12    public void run() {
13        try {
14            System.out.println("Thread processing file is : ");
15            FileInputStream in = new FileInputStream(filename);
16            MessageDigest sha = MessageDigest.getInstance("SHA-256");
17            DigestInputStream din = new DigestInputStream(in, sha);
18            while (din.read() != -1) ;
19            din.close();
20            byte[] digest = sha.digest();
21            StringBuilder result = new StringBuilder(filename);
22            result.append(": ");
23            result.append(DatatypeConverter.printHexBinary(digest));
24            System.out.println(result);
25        } catch (IOException ex) {
26            System.err.println(ex);
27        } catch (NoSuchAlgorithmException ex) {
28            System.err.println(ex);
29        }
30    }
31    public static void main(String[] args) {
32        System.out.println("args is : " + args);
33        for (String filename : args) {
34            DigestRunnable dr = new DigestRunnable(filename); //区别
35            Thread t = new Thread(dr); //区别
36            t.start();
37        }
38    }
39 }
```

运行参数:

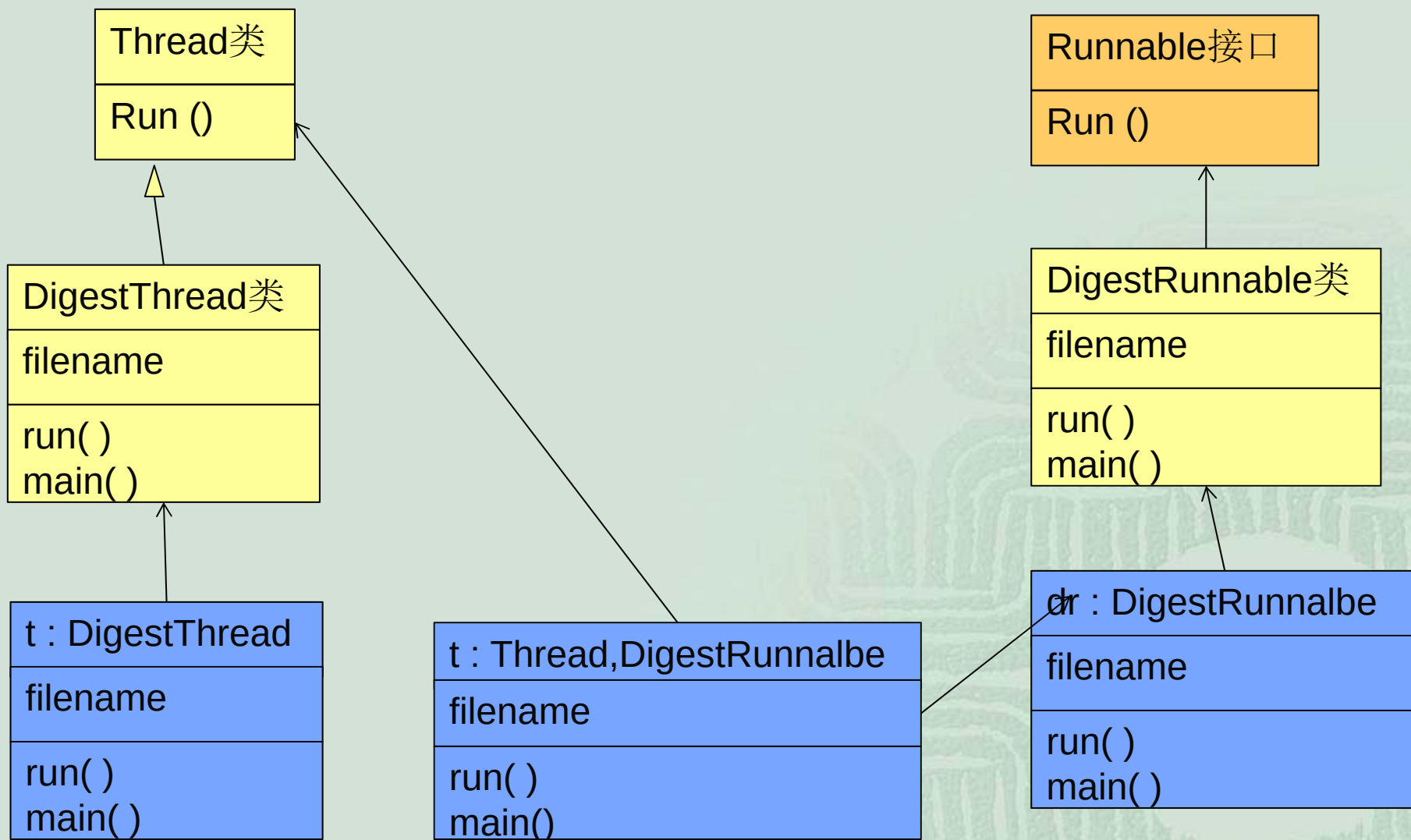
C:\temp\ch3data\digest\1.txt

C:\temp\ch3data\digest\build.xml

C:\temp\ch3data\digest\2.txt

分析

常用实现Runnable接口方式编写线程程序



在程序中如何从线程返回信息

- 线程的run()方法和start()方法不返回任何值。
- 直接使用存取方法返回线程任务的结果（有异常风险，程序执行结果不稳定）

演示编程示例ch3/returnInfo 3-3,3-4
ReturnDigestUserInterface.java



ReturnDigest.java

ReturnDigest.java

```
1 package ch3.returnInfo;
2
3 import java.io.*;
4 import java.security.*;
5
6 //用存取方法返回结果的线程 ex3-3
7 public class ReturnDigest extends Thread {
8
9     private String filename;
10    private byte[] digest;
11
12    public ReturnDigest(String filename) {
13        this.filename = filename;
14    }
15
16    @Override
17    public void run() {
18        try {
19            FileInputStream in = new FileInputStream(filename);
20            MessageDigest sha = MessageDigest.getInstance("SHA-256");
21            DigestInputStream din = new DigestInputStream(in, sha);
22            while (din.read() != -1) ; // read entire file
23            din.close();
24            digest = sha.digest();
25        } catch (IOException ex) {
26            System.err.println(ex);
27        } catch (NoSuchAlgorithmException ex) {
28            System.err.println(ex);
29        }
30    }
31
32    public byte[] getDigest() {
33        return digest;
34    }
35 }
```

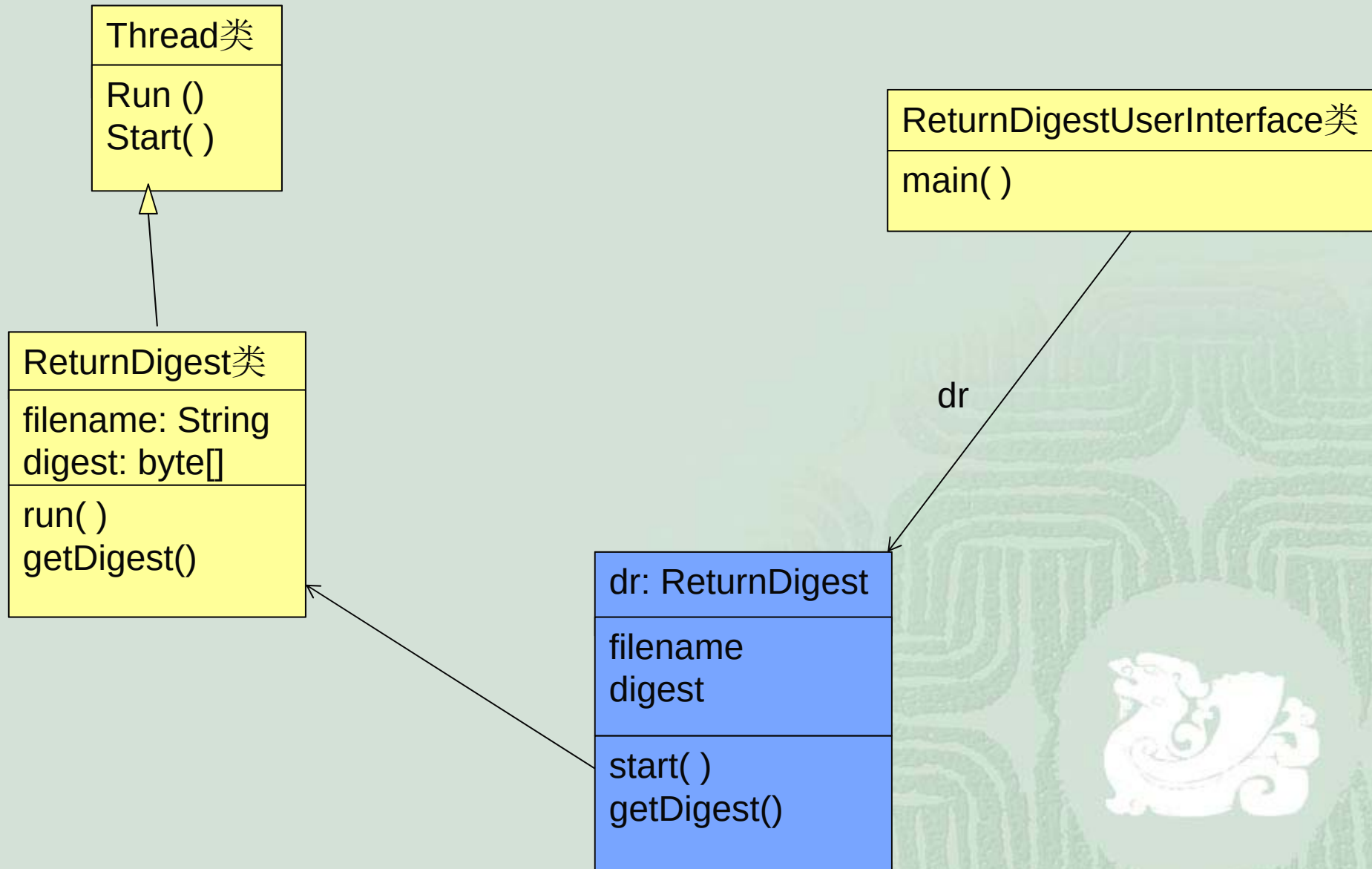
ReturnDigestUserInterface.java

```
1 package ch3.returnInfo;
2
3 import javax.xml.bind.*; // for DatatypeConverter
4
5 //用存取方法获得线程输出的主线程 ex3-4
6 public class ReturnDigestUserInterface {
7
8     public static void main(String[] args) {
9         for (String filename : args) {
10             // Calculate the digest
11             ReturnDigest dr = new ReturnDigest(filename);
12             dr.start();
13
14             // Now print the result
15             StringBuilder result = new StringBuilder(filename);
16             result.append(": ");
17             byte[] digest = dr.getDigest();
18             result.append(DatatypeConverter.printHexBinary(digest));
19             System.out.println(result);
20         }
21     }
22 }
```

ReturnDigestUserInterface.java



程序结构分析类图



程序运行不稳定的原因

- `dr.start()`启动的计算线程可能在`main()`方法调用`dr.getDigest()`之前结束，也可能还没有结束。
- 如果还没有结束，`dr.getDigest()`就会返回`null`，程序访问`digest`时就会抛出`NullPointerException`异常。

竞态条件：从线程返回信息，到底会得到正确的结果还是异常，或者被挂起，取决于程序生成了多少线程、系统的CPU和磁盘的速度、系统使用多少个CPU、JVM为不同线程分配时间所用的算法等因素。这些决定性因素称为竞态条件。



```

public static void main(String[] args) {
    ReturnDigest[] digests = new ReturnDigest[args.length];

    for (int i = 0; i < args.length; i++) {
        // 计算摘要
        digests[i] = new ReturnDigest(args[i]);
        digests[i].start();
    }

    for (int i = 0; i < args.length; i++) {
        while (true) {
            // 现在显示结果
            byte[] digest = digests[i].getDigest();
            if (digest != null) {
                StringBuilder result = new StringBuilder(args[i]);
                result.append(": ");
                result.append(DatatypeConverter.printHexBinary(digest));
                System.out.println(result);
                break;
            }
        }
    }
}

```

主线程占用CPU周期资源忙于轮询，做了大量不需要重的工作，浪费CPU周期。工作线程可工作的CPU周期很少。

回调法

- 让线程主动告诉主程序它何时结束（或能返回信息）。线程通过调用主类（即启动该线程的类）中的一个方法来通知主程序。称为**回调（callback）**。

演示静态回调编程示例ch3/callback 3-5,3-6
CallbackDigestUserInterface.java

相比于轮询法不会浪费太多CPU时间



CallbackDigest.java

CallbackDigest.java

```
1 package ch3.callBack;
2 import java.io.*;
3 import java.security.*;
4
5 public class CallbackDigest implements Runnable {
6
7     private String filename;
8
9     public CallbackDigest(String filename) {
10         this.filename = filename;
11     }
12
13     @Override
14     public void run() {
15         try {
16             FileInputStream in = new FileInputStream(filename);
17             MessageDigest sha = MessageDigest.getInstance("SHA-256");
18             DigestInputStream din = new DigestInputStream(in, sha);
19             while (din.read() != -1) ; // read entire file
20             din.close();
21             byte[] digest = sha.digest();
22             CallbackDigestUserInterface.receiveDigest(digest, filename);
23         } catch (IOException ex) {
24             System.err.println(ex);
25         } catch (NoSuchAlgorithmException ex) {
26             System.err.println(ex);
27         }
28     }
29 }
```

CallbackDigestUserInterface.java

CallbackDigestUserInterface.java

```
1 package ch3.callBack;
2
3 import javax.xml.bind.*; // for DatatypeConverter; requires Java 6 or JAXB 1.0
4
5 public class CallbackDigestUserInterface {
6
7     public static void receiveDigest(byte[] digest, String name) {
8         StringBuilder result = new StringBuilder(name);
9         result.append(": ");
10        result.append(DatatypeConverter.printHexBinary(digest));
11        System.out.println(result);
12    }
13
14    public static void main(String[] args) {
15        for (String filename : args) {
16            // Calculate the digest
17            CallbackDigest cb = new CallbackDigest(filename);
18            Thread t = new Thread(cb);
19            t.start();
20        }
21    }
22 }
```

回调法

- 演示讲解实例回调法编程示例。

ch3/instanceCallback 3-7,3-8

InstanceCallbackDigestUserInterface.java

实例回调编程方式比静态回调更灵活。

可应对多个类的对象实例想得到线程的计算结果的复杂场景。



InstanceCallbackDigest.java

InstanceCallbackDigest.java

```
1 package ch3.InstanceCallback;  
2 import java.io.*;  
3 import java.security.*;  
4 public class InstanceCallbackDigest implements Runnable {  
5     private String filename;  
6     private InstanceCallbackDigestUserInterface callback;  
7  
8     public InstanceCallbackDigest(String filename,  
9         InstanceCallbackDigestUserInterface callback) {  
10         this.filename = filename;  
11         this.callback = callback;  
12     }  
13  
14     @Override  
15     public void run() {  
16         try {  
17             FileInputStream in = new FileInputStream(filename);  
18             MessageDigest sha = MessageDigest.getInstance("SHA-256");  
19             DigestInputStream din = new DigestInputStream(in, sha);  
20             while (din.read() != -1) ; // read entire file  
21             din.close();  
22             byte[] digest = sha.digest();  
23             callback.receiveDigest(digest);  
24         } catch (IOException | NoSuchAlgorithmException ex) {  
25             System.err.println(ex);  
26         }  
27     }  
28 }
```



InstanceCallbackDigestUserInterface.java

```
InstanceCallbackDigestUserInterface.java ✕
2 import javax.xml.bind.*; // for DatatypeConverter; requires Java 6 or JAXB 1
3 public class InstanceCallbackDigestUserInterface {
4     private String filename;
5     private byte[] digest;
6     public InstanceCallbackDigestUserInterface(String filename) {
7         this.filename = filename;
8     }
9     public void calculateDigest() {
10         InstanceCallbackDigest cb = new InstanceCallbackDigest(filename, this);
11         Thread t = new Thread(cb);
12         t.start();
13     }
14     void receiveDigest(byte[] digest) {
15         this.digest = digest;
16         System.out.println(this);
17     }
18     @Override
19     public String toString() {
20         String result = filename + ": ";
21         if (digest != null) {
22             result += DatatypeConverter.printHexBinary(digest);
23         } else {
24             result += "digest not available";
25         }
26         return result;
27     }
28     public static void main(String[] args) {
29         for (String filename : args) {
30             // Calculate the digest
31             InstanceCallbackDigestUserInterface d
32                 = new InstanceCallbackDigestUserInterface(filename);
33             d.calculateDigest();
34         }
35     }
```



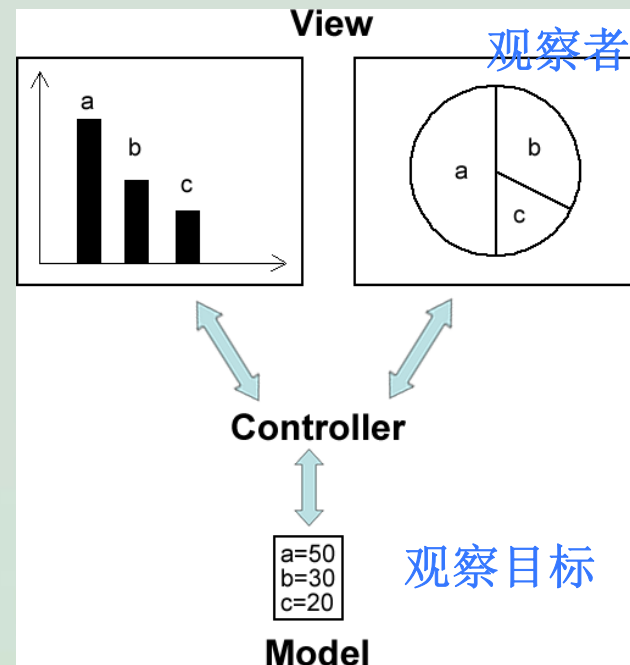
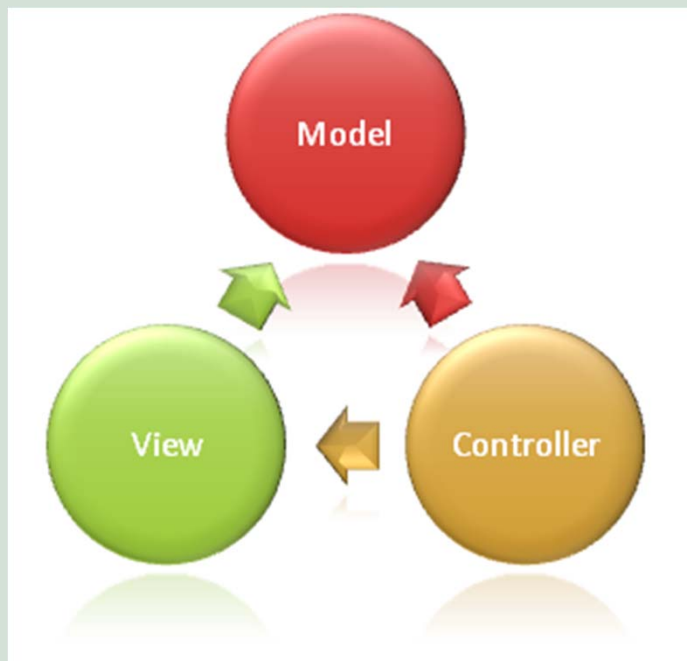
课后思考编程问题：

- 使用多线程编程，将文件安全算法摘要计算程序算出的文件安全算法摘要保存到对应的一个个文件中。例如，对于1.txt文件计算生成的安全算法摘要存入一个自命名为safeAbstrac1.txt中。请尝试改进文件安全算法摘要计算程序(DigestThread.java或DigestRunnable.java)，给出文件安全算法摘要保存程序。



深入扩展：观察者设计模式

➤ 观察者(Observer)设计模式



建立一种对象与对象之间的依赖关系，一个对象发生改变时将自动通知其他对象，其他对象将相应做出反应。

发生改变的对象称为观察目标。被通知的对象称为观察者，一个观察目标可以对应多个观察者，而且这些观察者之间没有相互联系，可以根据需要增加和删除观察者，使得系统更易于扩展。

谢谢

